Satelle Essentials

What Satelle is, and how to use it. Parts 1–4 are the working picture; part 5 is the reference.

1 The big picture

> Satelle is a **governance layer around the coding agent you already use** — a CLI, a set of hooks and a folder of rubrics. Work only happens inside a **story**, and a separate read-only reviewer approves each move. Tagline: *you raise the story, Satelle holds the gates.*



THE PARTS	WHAT IT DOES	WHERE
CLI	The record: stories, status, every verdict and what each cost. Moving a story is the only way work is sanctioned.	`satelle`
Hooks	Run before every edit and commit, and when a turn tries to end. They refuse when no story is in progress.	`.claude/settings.json`
Rubrics	Markdown handed to a fresh read-only judge at each point in the lifecycle. Versioned with your repo.	`.satelle/` (committed)
Database	Stories, statuses, verdicts and costs	`~/.satelle/<repo>-<hash>/` (never in git)
Dashboard `optional`	View stories across all your repos in a browser	`satelled` → `localhost:8787`

IT IS

- A **gatekeeper of status** — a story advances only through a reviewer's accept
- **Process as files** you can read, diff and override
- **Local-first**: no sign-in, no account, unless you bind a hosted project

IT ISN'T

- A **factory**. Satelle's docs call the lane your work runs through "the local factory" — that's the route, not the tool holding gates on it
- An **agent**, or a token seller. Gates run on your existing subscription
- A replacement for **git, PRs or CI**. It sits before them, on intent and scope

Authorship is not checked

Satelle does not guarantee a *human* wrote the story — nothing checks authorship. What it guarantees: the intent was **written down before the work**, **judged by an independent reviewer**, **frozen** once work began, and the resulting diff was **judged against that frozen text**. See 5B.

2 The everyday workflow

Four stages. Each move is guarded by more than one check; 5A names them.



STAGE	COMMAND	THE GUARD ASKS
Create	`satelle story create --title "…" --body "…" --category fix`	Is it well formed, correctly typed, and free of a prescribed solution?
Start	`satelle story estimate <id> …` `satelle story set <id> --status in_progress`	Is there an estimate, and is it clear enough to begin?
Done	`satelle story set <id> --status done`	Did the diff stay in scope, and does it deliver what the card promised?

Only one story may be in progress at a time. A rejected story stays where it is, with notes on what to fix.

Anatomy of a story

FIELD	FLAG	REQUIRED?	EXAMPLE
Title	`--title`	Yes	Add dark mode toggle to header
Body	`--body`	Yes	Users can switch themes and the choice is remembered
Category	`--category`	Yes	Picks the workflow — see below
Acceptance criteria	`--acceptance`	No — the readiness step authors them if you omit them	A numbered list, one testable line each

The card freezes once work starts

Title, body, acceptance criteria and category lock the moment a story leaves `backlog`, so an agent can't quietly weaken its own criteria to pass a gate. To change a frozen card use `satelle story amend <id> --reason "…"`, which is itself reviewed.

Picking a category

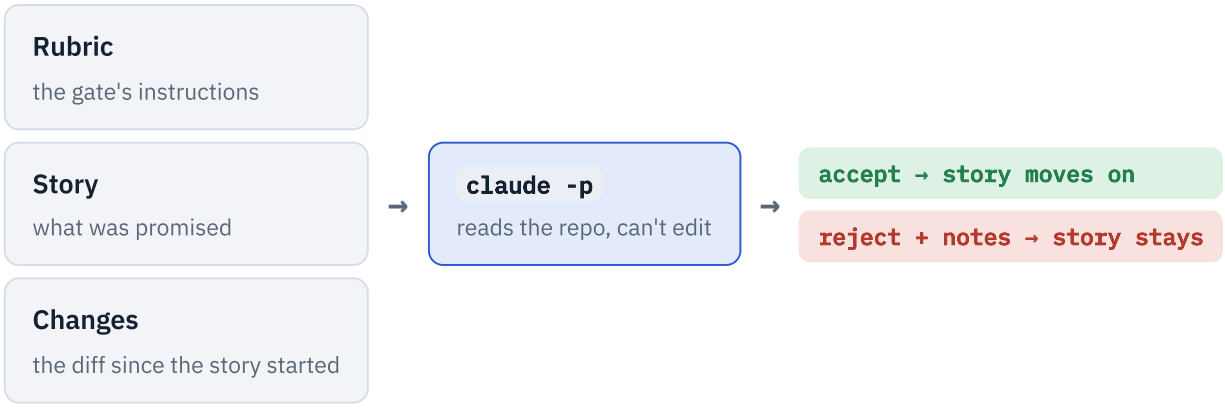
The category is the story's **type**, and it selects the workflow. These five cover almost everything; all 15 are in 5C.

USE	WHEN
`feature`	New behaviour that wasn't there before
`improvement`	Existing behaviour made better — reorganise, polish, extend
`fix`	Something is broken (`bug` , `bugfix` , `defect` all collapse here)
`docs`	Documentation only — a shorter route with one coded check at the close
`chore`	Housekeeping: dependency bumps, renames, tidying

Category is a type, never a surface: `ui` , `frontend` , `web` and `cli` are not categories — tag those `surface:ui` .

3 How gates work

A gate is a check on a move between stages. There are two kinds: one costs tokens and can be wrong, one can't.



KIND	HOW IT DECIDES	COSTS TOKENS?	DETERMINISTIC?
`LLM REVIEWER`	The rubric rides as a fresh agent's system prompt; it returns one JSON verdict	Yes — one agent run each	No — it is a judgement
`CODED CHECK`	A shell script in the skill file runs in the repo root. Exit 0 accepts, non-zero rejects with the output as notes	No	Yes

Fresh every time

A reviewer never saw the coding chat, so it judges only the result — and can't be argued into a pass.

Read-only

Granted `Read,Grep,Glob`, with Write, Edit and Bash denied. It judges; it never fixes.

Rubrics are files

Read one with `satelle doc get skills <name>`; override it by name in `.satelle/skills/`.

Which model a gate runs on

The reviewer binding ships with `model = ""`, so it resolves at dispatch. First one set wins:

binding's `model` › **step or gate `model`** › **your session's model** › **story creator's session** › **first of `[model_order]` — opus** › **CLI default**

Gates inherit your session, not Opus

Opus is the fallback when nothing earlier is set. In practice a gate runs on your session's model. Pin it with `model = "sonnet"` under `[reviewer]` in `.satelle/workflows/agents.toml`. No API keys are needed: gates use the login Claude Code already has. Check spend with `satelle story cost <id>`.

4 Setup & tips

One-time setup, then the common mistakes.

YOU First time in a repo

1. Install Satelle

```
curl -fsSL https://github.com/bobmc
```

2. Set the repo up

```
cd path/to/your-repo
satelle init
```

3. Check it's healthy

```
satelle doctor
```

4. Fill in

`.satelle/constitution.md`

Injected into every session. Left as the placeholder, every gate judges your repo blind.

5. Commit the setup yourself (it's not a story)

```
git add .gitignore .satelle && git
```

TEAMMATE After cloning

1. Install Satelle (same command)
2. Run

`satelle init`

in the clone

Recreates the hooks with paths for **their** machine. The hook commands hold absolute paths, so they can't be shared through git.

3. Start with a story

Hooks block Claude, not you

Commits from your own terminal aren't checked. The hooks only run inside the agent's session.

Leftover changes block every turn

An uncommitted file outside the exempt paths stops Claude finishing each reply. Commit it, revert it, or put it under a story.

Don't commit `.claude/settings.json`

It holds your machine's absolute paths. Gitignore it unless the team shares a layout.

On a Mac, run the dashboard yourself

`satelle service install` needs systemd. Use `satelled --addr 127.0.0.1`.

The dashboard hides closed stories

It opens filtered to `status:active`. Clear the chip, or use `?stories=status:all`, to see finished work.

Satelle's own rules are locked

Everything under `.satelle/` is exempt from the edit gate but locked to the `substrate` lane, so the agent can't rewrite the rubrics that judge it.

5 The details

Reference for parts 1–4. `satelle workflow validate` prints the same map for your own repo.

5A Every gate, transition by transition

The **working lane** — the route for `feature`, `improvement`, `fix`, `chore`, `refactor`, `test`, `tooling`, `infrastructure`, `architecture` and `substrate`.

1 · Create

`draft → backlog`

2 CHECKS · 1 AGENT RUN

CHECK	KIND	WHAT IT ACTUALLY JUDGES	REJECTS?
structure check	CODED	Three things only: a specific title that names the change, a body stating the goal, and a non-empty category. Acceptance criteria are *not* part of this.	YES
`satelle-story-create-review`	LLM	Coherence (one real outcome, not two goals stapled together) · acceptance criteria against the goal · that the card states desired *behaviour* and does not prescribe a function, type, file or mechanism · that it advances the constitution and contradicts none of it · a dead premise · category and tags.	YES

Nothing is saved until both pass. Switch the LLM half off with `[review] gate_create = false`; the structure check always runs.

2 · Start work

`backlog → in_progress`

3 CHECKS · 2 AGENT RUNS

CHECK	KIND	WHAT IT ACTUALLY JUDGES
`satelle-estimate-actual-review`	CODED	An estimate tag must exist (`satelle story estimate <id> --fresh-input <n> --ou` It then compares your token estimate against comparable fi stories and rejects one outside tolerance, naming the ids an Too few comparables → accepts with a note. A dollar figure estimate.
`satelle-story-intent-review`	LLM	Is the story as presented well formed enough to begin: a titl behaviour card, and testable criteria — from the story, or fro acceptance criteria when the story has none. It does not rew story.
`satelle-step-summary`	LLM	Writes a short recap of the transition into the ledger. Observ narrates only.

Outside the gates, two things happen here: Satelle records an **engagement baseline** (git HEAD plus a dirty flag) that every later scope check measures from, and the story card **freezes**.

3 · Close

`in_progress → done`

6–7 CHECKS · 4–5 AGENT RUNS

CHECK	KIND	WHAT IT ACTUALLY JUDGES	REJECTS?
`satelle-workflow-change-review`	LLM	Route edits: where a gate is bound, whether it over-fires, whether skills and obligations still resolve, and that park / cancel / recover survived. **Fast-accepts** when the diff touches no workflow file — which is most stories.	YES
`satelle-story-scope-review`	LLM	Boundedness: does the diff since the engagement baseline stay within *this* story's acceptance criteria? The gate that rejects bundling a sibling fix into the slice.	YES
`satelle-story-done-review`	LLM	The spine gate. It names the **behaviour** the card promises and checks the change delivers it — recorded as criterion 0 — then walks each numbered criterion against the evidence. **If the behaviour fails it rejects even when every criterion passes.**	YES
`satelle-estimate-actual-review`	CODED	Prints the estimate beside the measured token actuals, for the record.	NEVER
`satelle-epic-branch-check`	CODED	Only bites inside a container: a child with no `lane:trunk` tag closes only once its slice is on a pushed `epic/<id>` branch not yet on trunk. A story in no container passes untouched.	IN EPICS
`satelle-instruction-change-review`	LLM	**Conditional.** A trigger script fires it only when the diff adds a new instruction file or changes at least 5 words in `.satelle/skills/`, `.satelle/principles/` or `constitution.md`. When it fires it asks three things of the new text: does it help produce	WHEN TRIGGERED

CHECK	KIND	WHAT IT ACTUALLY JUDGES	REJECTS?
		accurate readable code, is it about Satelle and this project, does it duplicate context already injected.	
`satelle-step-summary`	LLM	The closing recap into the ledger.	NEVER

The first three run **serially**, not in parallel — the shipped step sets `parallel = 0`. A rejection at scope costs the done review too.

Side exits

`park, abandon, amend`

1 CHECK EACH

MOVE	GATE	ASKS
→ `blocked`	`satelle-story-blocked-review`	Is the blockage real? A `blocked-triage` advisor runs alongside it.
→ `cancelled`	`satelle-story-cancel-review`	Is abandoning this legitimate? Available from backlog, in_progress and blocked.
Amend the card	`satelle-story-amend-review`	Is the amendment sound? The sanctioned way to change a frozen story rather than cancelling and re-raising, which retires the id and its ledger.

What a story costs

A `feature` or `fix` that passes first time is **about seven agent runs** — one at create, two at start, four at close — plus four coded checks that cost nothing. Every rejection re-runs that edge's gates. Pin `[reviewer] model` below your session's.

5B Who may raise a story

Nothing requires a human to write the story. The agent can call `satelle story create` and start work itself, and the shipped `satelle-agent-goals` principle tells it to: a blocked agent must "raise the work as a story", and a declared transition is authorised by the workflow "without pausing to ask permission".

CONCERN	WHAT ACTUALLY HOLDS
The agent writes the story it then implements	Allowed by design. The defence is that the **create review is a different, read-only agent** that never saw your conversation, and it rejects a card that prescribes the solution, is incoherent, or contradicts the constitution. It cannot know what you *meant* — only whether the card is sound.
The agent softens the criteria to pass its own gate	Blocked. The card **freezes** at `in_progress`; changing it afterwards goes through `satelle-story-amend-review` and leaves a ledger entry.
The agent does work no story covers	Blocked by the edit and commit hooks, and by the stop check at the end of every turn.
I want raising and implementing on different models	`[gate] no_implement_models` — a glob of caller models the edit hook refuses outright, with `no_implement_message` as the deny text and `no_implement_exempt_paths` / `_globs` as carve-outs. Ships unset. Inspect a decision with `satelle hook explain --payload`.
I want to sign off before work starts	**No setting exists.** The lever is `.satelle/constitution.md`, injected into every session: state there that the agent drafts and presents a story for your approval, and never moves it to `in_progress` unprompted. Behavioural, not enforced — treat it as such.

5C All 15 categories

The category selects the route, so it changes which gates you pay for.

GROUP	CATEGORIES	USE IT FOR	ROUTE AND CLOSE
Working lane	`feature` `improvement` `fix` `chore` `refactor` `test` `tooling` `infrastructure` `architecture`	Everyday code changes, from new behaviour to CI and dependency bumps	backlog → in_progress → done, with the full ladder in 5A
Docs	`docs`	Documentation only	Same three stages, but the close is **one coded check** (`satelle-docs-only-check`): the slice stayed inside doc paths. No LLM reviewer, and no estimate gate.
Satelle itself	`substrate` `agents-repair`	Changing the rules under `.satelle/` · repairing `agents.toml`	`substrate` runs the working lane and is the **only** lane allowed to edit locked rule files. `agents-repair` closes on one coded check that nothing but `agents*.toml` changed — no LLM, since the bindings being repaired are what would dispatch it.
Containers	`epic-parent` `parent`	Grouping several child stories under one theme	A different route: backlog → **ready** (sequences the children) → **merging** (lands their branches) → done, gated by `satelle-story-done-review` + `satelle-epic-merged-check`. Closes only when every child is done or cancelled.
Special	`empty`	A dry run with no product content, to prove a workflow	Takes the working lane, but is skipped by stats and estimate comparables.

Synonyms collapse: `bug`, `bugfix`, `defect` → `fix`; `infra` → `infrastructure`. An unknown category only warns by default (`[categories] enforce = "warn"`); add your own with `[categories] extra`. **Tasks are a separate kind**, not a category: re-runnable jobs under `.satelle/tasks/`, gated by `satelle-task-validate-before-review` on entry and `…-after-review` at the close.

5D Reading and changing any of this

Every rule is configuration, not code.

I WANT TO…	RUN OR EDIT
See every gate in my repo, and its model	`satelle workflow validate`
Read a gate's actual rubric	`satelle doc get skills satelle-story-done-review`
Override a rubric	Drop a file of the same name in `.satelle/skills/` — under a `substrate` story
Skip the create review	`[review] gate_create = false`
Put one gate on a different model	Add a second `[name]` binding and point the step or gate at it
Edit a folder without a story	Add the prefix to `[gate] edit_exempt_paths`
Make tiny fixes without a full story	List your product paths in `[fix_lane] product_surface`, then `satelle fix claim`. Closed until you do.
Work several children at once	`[engagement] parallel = "epic"` plus `satelle story worktree`
Turn it off in Claude Code	Remove the `hooks` block from `.claude/settings.json`

Built-in guides

`satelle help reviewer-checks` · `agent-dispatch` · `workflows` · `create-story` · `substrate` · `fix-lane` · `session`

Based on satelle 0.0.894 (built 2026-11-05), its built-in help, and the shipped default workflow as reported by `satelle workflow validate`. It's an early release, so commands and defaults may change — and a repo that authors its own `done.toml` / `step.toml` changes the ladder in 5A.